

PRD: Autonomous Multi-Market Prediction, News, Copy-Trader, and Quant Research Engine

Research snapshot: September 4, 2026

Target operator: Claude Code or equivalent terminal-capable coding agent

Deployment model: local machine, terminal-only, backend-only, continuously running

Primary objective: build a research-and-paper-trading laboratory that consumes real production market data, runs many independent strategies against the same data, measures them honestly, kills weak strategies, promotes durable ones, and eventually combines the best performers into a tightly risk-controlled live strategy.

The core principle of this PRD is the user's stated rule: **"If it isn't broke, don't fix it."** That does **not** mean blindly copying historical winners. It means proven ideas, public winning traders, academically documented effects, and prominent open-source implementations are treated as *starting hypotheses*, then subjected to the same point-in-time, fee-aware, latency-aware, out-of-sample paper-trading system before being trusted with the initial \$50 live bankroll.

Product mandate and operating philosophy

The system shall be named provisionally **MarketLab**. Claude may rename it, but every subsystem described here must remain represented in the implementation.

MarketLab is not one trading bot. It is an **experimental trading operating system**.

At any moment it may be running dozens or hundreds of independent strategy variants across:

Domain	Initial universes
Prediction markets	Kalshi production markets, Polymarket global read-only markets, Polymarket US where available
Crypto-event markets	BTC direction/range markets at approximately 1-minute, 5-minute, 15-minute, 1-hour, daily horizons wherever equivalent markets exist
Sports	NFL, NBA, MLB, NHL, college sports and other liquid event contracts
Politics	elections, approval, nominations, legislation, political-event markets
Economics	CPI, Fed, jobs, GDP, rates, recession and related contracts
Stocks	liquid U.S. equities, ETFs, event-sensitive stocks
Social/event signals	Donald Trump/public-political statements, X posts, news shocks, company mentions

Domain	Initial universes
Public trading activity	SEC filings and legally usable disclosed transaction datasets
Copy trading	public Polymarket leaderboard wallets and category specialists
Cross-market	equivalent or related contracts whose price discrepancies can be measured
Weather	weather prediction contracts backed by NWS/NOAA data where applicable

Kalshi is particularly appropriate for this architecture because its production Trade API exposes market data and authenticated trading over REST and WebSocket, and its documentation explicitly separates production and demo environments and credentials. The current recommended production REST endpoint is the external Trade API host; production WebSocket data is available through the corresponding external WebSocket host. ¹

Polymarket currently exposes separate Gamma, CLOB, Data, Relayer and realtime interfaces. Gamma is intended for market/event discovery, the CLOB for live market state and order operations, and the Data API for account and activity information. Its realtime stack includes market, user and sports channels. ²

There is an important September 2026 constraint for this specific project: **global Polymarket currently lists the United States as close-only on both its frontend and API, meaning new global CLOB orders should not be attempted from the user's U.S. location.** Polymarket provides an official geoblock endpoint and instructs applications to check eligibility before submitting orders. Therefore, MarketLab shall use global Polymarket as a **read-only price, market, leaderboard and trader-intelligence source**, while any U.S. Polymarket execution must go through the separate official Polymarket US product/API, subject to account eligibility. The project must never attempt to bypass a geoblock. ³

That distinction changes the architecture:

```

Kalshi Production
  market data -> YES
  local paper -> YES
  live orders -> optional, gated

Polymarket Global
  market data -> YES
  trader data -> YES
  leaderboard -> YES
  local paper -> YES
  live orders -> NO from U.S. under current global restrictions

Polymarket US
  market data -> YES where available

```

```
local paper -> YES
live orders -> optional, gated and eligibility-dependent
```

The official Polymarket US Python SDK presently supports unauthenticated event, market, book, BBO, search, series and sports calls, plus authenticated limit orders, cancellations, positions, balances and private realtime WebSocket activity. It uses Ed25519 authentication. ⁴

Paper trading must be local, not dependent on demo balances. Kalshi's demo environment is useful for validating authentication and order API mechanics, but Kalshi itself notes that demo uses mock funds and its prices/behavior may differ from production. The primary research environment should therefore consume **production books and production trades but route proposed orders into MarketLab's own PaperBroker.** ⁵

The resulting system shall enforce four operating modes:

```
DATA_ONLY
  ingest and record data
  never create orders

BACKTEST
  replay stored historical events deterministically

PAPER
  consume live production data
  simulate all execution locally
  never call a venue create-order endpoint

LIVE
  production trading
  disabled by default
  explicit credentials + eligibility + risk gates required
```

LIVE must be impossible to activate accidentally.

A strategy shall not know whether it is running in historical, paper, or live mode. Strategies emit standardized **OrderIntent** or **ProbabilityForecast** objects to a broker interface. The broker decides whether they become simulated orders or real orders.

That separation is non-negotiable because the final goal is to prevent the common situation where a backtest implementation and a live implementation behave differently.

The \$50 experiment. Live capital begins as one explicitly tracked \$50 bankroll. Paper research may create many virtual \$50 strategy sleeves so that strategies can be compared on equal starting capital.

Each paper sleeve follows this lifecycle:

```
INITIAL_CAPITAL = $50
NO_REPLENISHMENT_WITHIN_RUN = true
CAN_REACH_ZERO = true
RESTART_CREATES_NEW_EXPERIMENT_ID = true
OLD_RESULTS_NEVER_DELETED = true
```

Thus, when the user says “start with \$50, die with it, or create more to live,” MarketLab interprets that scientifically:

A strategy may destroy its original \$50 experimental sleeve. It does not receive artificial rescue money. A new \$50 run is a **new cohort**, not a continuation that hides the prior failure.

Real money is stricter. A \$50 live bankroll may decline, but MarketLab shall not martingale, average down automatically merely because a position lost, borrow, add leverage, or refill the bankroll without recording that addition as a new capital epoch.

The target is not “maximum backtest return.” The target is:

Maximum repeatable out-of-sample expected value after fees, spread, latency, failed fills, stale information and correlated risk.

A strategy with a 500% historical backtest but no surviving forward edge is a loser. A strategy making 5% with low drawdown and repeatable execution may be a winner.

The system shall preserve every losing experiment. No result may be manually deleted because it makes the overall statistics look bad.

Each experiment must have an immutable identifier derived from at least:

```
strategy_name
strategy_version
git_commit
parameter_hash
market_universe
venue
data_version
execution_model_version
feature_version
LLM_model_id, if applicable
prompt_hash, if applicable
start_timestamp
starting_bankroll
```

A strategy cannot be silently modified under the same experiment identity.

Research findings, proven ideas, APIs, and open-source foundations

The research strongly supports building a **strategy tournament**, not searching for one magical algorithm.

Several trading ideas have credible historical or theoretical foundations, but none deserves a “guaranteed profitable” label.

Time-series momentum is one of the stronger academically documented baselines. Moskowitz, Ooi and Pedersen studied 58 liquid equity-index, currency, commodity and bond futures/forwards and documented significant time-series momentum in the historical sample. That makes momentum a legitimate baseline to test on BTC/stock-derived signals, but not evidence that the same coefficients will automatically work on a binary prediction contract in 2026. ⁶

Pairs/statistical-arbitrage trading also deserves a baseline arm. Gatev, Goetzmann and Rouwenhorst studied a systematic pairs methodology on U.S. equities over a long historical dataset and reported meaningful historical excess returns after conservative transaction-cost assumptions. Again, this is evidence that mean-reverting relative-value strategies are worth testing, not proof of future profitability. ⁷

For passive market making, the Avellaneda-Stoikov framework remains a foundational model for placing bid and ask quotes while managing inventory risk. MarketLab should adapt its concepts—not copy its assumptions blindly—to bounded \$0–\$1 prediction contracts. ⁸

Prediction-market arbitrage is sufficiently real to merit a dedicated strategy family: recent academic work has explicitly examined arbitrage in prediction markets including Polymarket. At the same time, research challenging persistent-arbitrage claims shows why MarketLab must distinguish genuine executable opportunities from apparent spreads caused by asynchronous timestamps, liquidity, contract-definition mismatches, or optimistic execution assumptions. ⁹

The codebase should therefore classify evidence rather than claim everything is “proven”:

Evidence class	Meaning	Examples
A	credible academic empirical evidence	time-series momentum, pairs trading
B	established quantitative theory/mechanism	inventory-aware market making, probability arbitrage
C	prominent open-source implementation pattern	Hummingbot, Freqtrade, NautilusTrader
D	directly observable public trader behavior	Polymarket leaderboard/copy experiments
E	plausible market microstructure hypothesis	order-book imbalance, short-horizon reversal
F	AI/social hypothesis	LLM news probability, X sentiment

Evidence class	Meaning	Examples
G	viral/unverified claim	"turned \$100 into \$1m" posts

Class G strategies may be investigated but can never skip validation.

Prominent open-source references.

Hummingbot is particularly relevant to the exchange/market-making side of this project. Its GitHub repository currently has roughly 19.8k stars and describes itself as a framework for creating and deploying high-frequency trading bots. It should be studied for connector organization, order lifecycle management, market-making abstractions and operational patterns—not blindly imported wholesale. ¹⁰

Freqtrade provides open-source crypto bot infrastructure with backtesting, strategy optimization, money management and dry-run/live patterns, and has a separate public strategies repository. It is a useful reference for strategy configuration, hyperparameter experiments, dry-run discipline and reporting. ¹¹

Backtrader remains a prominent historical Python backtesting library; public GitHub data has shown more than 22k stars, although its architecture is older than the event-driven design desired here. It is useful as an API-design reference but should not be selected automatically as the project's core. ¹²

NautilusTrader is a particularly strong architectural reference because it explicitly unifies research, deterministic simulation and live execution inside one event-driven, multi-asset, multi-venue engine with a Rust core and Python strategy/control layer. Recent project documentation also includes Polymarket integration work. Claude should inspect its current adapter compatibility before deciding whether to embed it or merely borrow its design principles. ¹³

For the AI-agent layer, TradingAgents is a very prominent open-source multi-agent finance project. Its current project describes specialized fundamental, sentiment, technical, research, trading and risk agents, and its August 2026 release notes specifically mention point-in-time/look-ahead corrections—exactly the type of problem MarketLab must treat seriously. Third-party star trackers have reported the project in the tens of thousands of GitHub stars, but popularity is not evidence of profitable live trading. ¹⁴

FinGPT is another legitimate research reference. The AI4Finance project describes it as an open-source financial-language-model ecosystem and reports more than 20k GitHub stars on its project site; its associated research focuses on financial data acquisition, cleaning, preprocessing and financial NLP. Use its ideas for news/sentiment processing, not as permission for an LLM to directly gamble capital. ¹⁵

The `ai-hedge-fund` GitHub project is prominent and useful for agent orchestration patterns, but its own README explicitly calls itself a proof of concept for educational use and says it is not intended for real trading. MarketLab may borrow architecture, never performance claims. ¹⁶

Polymarket itself maintains an official Agents repository for autonomous AI experimentation, which is valuable as a platform-specific reference. ¹⁷

For current Polymarket implementation, Claude must **not** build against old tutorials that assume the original CLOB V1 stack. Polymarket migrated to CLOB V2 in April 2026, and the older `py-clob-client` repository has since been archived and marked no longer functional. New development should prefer Polymarket's current official SDKs. ¹⁸

The project should maintain a file called:

```
docs/research_registry.yaml
```

with entries such as:

```
strategies:
  time_series_momentum:
    evidence: A
    source_family: academic
    implementation_status: required

  pairs_mean_reversion:
    evidence: A
    source_family: academic
    implementation_status: required

  avellaneda_stoikov_binary:
    evidence: B
    source_family: academic_theory
    implementation_status: required

  cross_venue_binary_arbitrage:
    evidence: B
    implementation_status: required

  polymarket_copy_trader:
    evidence: D
    implementation_status: required

  book_imbalance:
    evidence: E
    implementation_status: required

  llm_news_probability:
    evidence: F
    implementation_status: required

  viral_wallet_clone:
```

```
evidence: G
implementation_status: research_only
```

Primary APIs.

The current endpoint inventory Claude should encode through adapters is:

```
KALSHI PRODUCTION REST
https://external-api.kalshi.com/trade-api/v2

KALSHI PRODUCTION WS
wss://external-api-ws.kalshi.com/trade-api/ws/v2

KALSHI DEMO REST
https://external-api.demo.kalshi.co/trade-api/v2

KALSHI DEMO WS
wss://external-api-ws.demo.kalshi.co/trade-api/ws/v2

POLYMARKET GLOBAL GAMMA
https://gamma-api.polymarket.com

POLYMARKET GLOBAL CLOB
https://clob.polymarket.com

POLYMARKET GLOBAL DATA
https://data-api.polymarket.com

POLYMARKET GLOBAL LEADERBOARD
https://data-api.polymarket.com/v1/leaderboard

POLYMARKET GLOBAL GEOBLOCK
https://polymarket.com/api/geoblock

OLLAMA DEFAULT LOCAL API
http://localhost:11434/api

SEC EDGAR JSON
https://data.sec.gov
```

These hosts and roles are documented by the respective platforms as of the research date. ¹⁹

Kalshi authentication uses an API key ID plus asymmetric signing headers; the project's key material must stay outside Git and outside logs. ²⁰

Kalshi rate limiting is dynamic enough that Claude should not hard-code a single universal request budget. Kalshi documents token-based costs and provides an endpoint for current endpoint costs, so the adapter must have a rate-limit manager and honor current server behavior. ²¹

Polymarket similarly documents per-service API limits; the ingestion layer should batch where supported, subscribe to WebSockets where appropriate, and react to 429 responses rather than polling unnecessarily. ²²

Data sources beyond trading venues.

GDELT should be the zero/low-cost global news backbone. Its ecosystem provides realtime machine-readable news/event APIs, and its event database uses hundreds of event categories with frequent updates. GDELT's Context API can also return sentence-level context around queries. ²³

SEC EDGAR should drive public-company filing signals. The SEC's `data.sec.gov` APIs require no API key, provide submissions/XBRL data, and are updated throughout the day as filings disseminate. That makes 8-Ks, 10-Qs, 10-Ks and related filing events viable stock-signal inputs. ²⁴

Alpaca can serve as an optional normalized U.S. stock/news data provider. Its WebSocket market-data products cover real-time stock feeds, and its market-data stack also exposes news streams. ²⁵

Alpha Vantage can be an inexpensive fallback for historical/reference market data and news/sentiment, but its normal free allowance is too constrained to be the engine's high-frequency primary feed. ²⁶

FRED should provide macroeconomic series and ALFRED-style point-in-time data where relevant; the St. Louis Fed provides programmatic economic-data APIs, including a newer bulk-oriented API version. ²⁷

The National Weather Service API should back U.S. weather-market strategies because it exposes forecasts, observations and alerts programmatically. ²⁸

For sports comparison, The Odds API is a reasonable optional external reference feed; it offers bookmaker odds across numerous sports and a small free allowance. It must be used as a comparison signal, not confused with exchange-executable liquidity. ²⁹

X's official Developer Platform should be preferred over unauthorized scraping for the Twitter/X tracker. The current platform advertises usage-based API access, so the adapter must have a cost budget and degrade gracefully when X credentials are absent. ³⁰

Bluesky is an unusually useful secondary social source because its public firehose can be consumed over a realtime stream without relying on a single curated keyword search API. ³¹

Public-trader discovery is now much easier than relying on third-party websites.

Polymarket currently documents an official trader leaderboard endpoint. It can rank traders by P&L or volume and filter by categories including politics, sports, crypto, economics, finance, tech, weather and

others, over day/week/month/all-time periods. Its response includes a profile wallet, username, volume, P&L and potentially an X username. ³²

The API also exposes public user activity, positions, closed positions and user/market trade history. That means MarketLab can create a defensible **dynamic trader-discovery system** rather than hard-coding whoever happens to be famous today. ³³

This is preferable to blindly selecting a viral “AI bot” account. Search results contain spectacular public claims such as tiny starting balances becoming millions, but those claims are frequently posted in social or promotional content rather than independently audited records. Such accounts should enter the candidate database as `UNVERIFIED_SOCIAL_CLAIM`, then be validated using public trade and position history before they can influence any strategy. ³⁴

Therefore, the PRD deliberately does **not** hard-code a September 4 leaderboard wallet and call it “the winner.” Rankings change. The system shall retrieve and snapshot the actual winners automatically every day.

The copy discovery job shall query combinations such as:

```
OVERALL / DAY / PNL
OVERALL / WEEK / PNL
OVERALL / MONTH / PNL
OVERALL / ALL / PNL

CRYPTO / DAY / PNL
CRYPTO / WEEK / PNL
CRYPTO / MONTH / PNL

SPORTS / DAY / PNL
SPORTS / WEEK / PNL
SPORTS / MONTH / PNL

POLITICS / WEEK / PNL
POLITICS / MONTH / PNL

ECONOMICS / WEEK / PNL
FINANCE / WEEK / PNL
WEATHER / WEEK / PNL
```

Store the top 50 from each snapshot.

A trader becomes a candidate only after the system computes:

```
discovery_date
rank_at_discovery
```

```
category
day_pnl
week_pnl
month_pnl
all_time_pnl
reported_volume
number_of_markets
number_of_observed_trades
median_trade_size
position_concentration
resolved_win_rate
realized_pnl
estimated_roi
largest_loss
largest_win
max_observed_drawdown
category_specialization
median_holding_time
turnover
recent_performance_slope
```

Then the key experiment begins: **forward performance after discovery**.

This prevents MarketLab from fooling itself by selecting today's winner and backtesting that same wallet's past.

Backend architecture, market normalization, and local paper exchange

MarketLab shall be a Python-first asynchronous backend with no graphical interface.

Recommended baseline:

```
Python 3.12+
uv for environment/dependency management
asyncio
httpx
websockets
pydantic
typer
rich
orjson
polars
pyarrow
duckdb
```

```
sqlite in WAL mode
numpy
scipy
statsmodels
scikit-learn
tenacity
feedparser
trafilatura
pytest
ruff
mypy
```

Claude may change individual dependencies where technically justified, but not the architecture.

Use `Decimal` or integer tick units for money/probability values where exact price arithmetic matters. Do not let binary contract accounting depend on floating-point equality.

The repository shall resemble:

```
marketlab/
├─ README.md
├─ pyproject.toml
├─ uv.lock
├─ .env.example
├─ .gitignore
├─ Makefile
├─ configs/
│   ├─ default.yaml
│   ├─ paper.yaml
│   ├─ live.example.yaml
│   ├─ universes.yaml
│   ├─ strategies.yaml
│   ├─ risk.yaml
│   ├─ copy_traders.yaml
│   └─ sources.yaml
├─ docs/
│   ├─ PRD.md
│   ├─ architecture.md
│   ├─ research_registry.yaml
│   ├─ live_safety.md
│   └─ data_dictionary.md
├─ marketlab/
│   ├─ cli.py
│   ├─ settings.py
│   ├─ logging.py
│   └─ clock.py
```

- └─ core/
 - └─ events.py
 - └─ instruments.py
 - └─ orders.py
 - └─ portfolio.py
 - └─ strategy.py
 - └─ broker.py
 - └─ probability.py
- └─ adapters/
 - └─ kalshi/
 - └─ polymarket_global/
 - └─ polymarket_us/
 - └─ alpaca/
 - └─ sec/
 - └─ gdelt/
 - └─ x/
 - └─ bluesky/
 - └─ fred/
 - └─ weather/
 - └─ sports_odds/
- └─ storage/
 - └─ schema.py
 - └─ state.py
 - └─ parquet.py
 - └─ migrations/
- └─ execution/
 - └─ paper_broker.py
 - └─ fill_models.py
 - └─ kalshi_live.py
 - └─ polymarket_us_live.py
 - └─ risk_gateway.py
- └─ matching/
 - └─ cross_venue.py
 - └─ resolution_rules.py
 - └─ semantic_match.py
- └─ signals/
 - └─ orderbook.py
 - └─ technical.py
 - └─ news.py
 - └─ social.py
 - └─ filings.py
 - └─ copy_trader.py
 - └─ sports.py
- └─ strategies/
 - └─ baseline_market.py
 - └─ momentum.py
 - └─ mean_reversion.py
 - └─ market_maker.py

Do not start with an unnecessarily complicated distributed infrastructure.

Use:

SQLite WAL:

- experiments
- orders
- simulated fills
- positions
- balances
- strategy state
- trader registry
- market registry
- alerts
- service checkpoints

Append-only Parquet:

- order-book snapshots/deltas
- trades
- prices
- market metadata history
- news
- social posts
- external reference prices
- strategy forecasts

DuckDB:

- analytics across Parquet
- research queries
- leaderboard reports

If load eventually exceeds this design, migration to PostgreSQL/NATS/Kafka can be a later experiment. Do not add infrastructure before the local machine needs it.

Unified market object.

Every event contract becomes:

```
NormalizedMarket(  
    canonical_id,  
    venue,  
    venue_market_id,  
    event_id,  
    title,  
    description,
```

```
    resolution_rules,  
    resolution_source,  
    category,  
    subcategory,  
    outcome_type,  
    yes_symbol,  
    no_symbol,  
    open_time,  
    close_time,  
    expected_resolution_time,  
    timezone,  
    tick_size,  
    min_order,  
    max_payout_per_contract,  
    status,  
    fees,  
    liquidity,  
    volume,  
)
```

Prices normalize to:

```
0.00 <= probability_price <= 1.00
```

A Kalshi price expressed in cents internally becomes a decimal probability price. Do not mix cents, dollars and probabilities in strategy code.

Canonical matching is one of the most important modules in the entire project.

Two markets with similar English titles are not necessarily the same bet.

Cross-venue matching must compare:

```
subject  
outcome  
threshold  
measurement  
time window  
timezone  
resolution authority  
cancellation rule  
postponement rule  
inclusion/exclusion language  
settlement timing
```


Example conceptual failure:

```
Venue A:  
"Will BTC be above $100k at 5 PM ET?"  
  
Venue B:  
"Will BTC reach $100k before 5 PM ET?"
```

These are **not equivalent**.

The matcher shall first use deterministic token/entity/time extraction. An embedding or local LLM may propose candidates. Then a deterministic resolution-rules validator must approve them.

Store:

```
match_confidence  
same_outcome_boolean  
rule_diff  
time_diff  
resolution_source_diff  
human_review_required
```

A cross-venue "arbitrage" strategy may only operate automatically where `same_outcome_boolean=true` and rule confidence exceeds a stringent threshold.

The PaperBroker is the heart of trustworthy research.

The system shall not use:

```
IF last_price <= my_limit:  
    fill me
```

That creates fantasy profits.

Market orders must walk the **real visible production book at simulated arrival time**.

For a buy:

```
fill first against best ask  
then next ask  
then next
```

```
until requested quantity is filled  
or visible liquidity exhausted
```

Record:

```
decision_timestamp  
simulated_network_send_timestamp  
simulated_exchange_arrival_timestamp  
book_timestamp_used  
average_fill_price  
worst_fill_price  
filled_quantity  
unfilled_quantity  
slippage  
fee
```

Paper latency must be configurable:

```
execution:  
  signal_to_order_ms: 100  
  network_latency_ms: 75  
  processing_latency_ms: 25
```

Then run sensitivity variants:

```
50 ms  
100 ms  
250 ms  
500 ms  
1 s  
5 s  
30 s
```

This is especially important for copy trading and apparent arbitrage.

Limit-order simulation must be conservative.

Support at least three models:

```
TOUCH  
  optimistic research diagnostic only
```

TRADE_THROUGH

fill only when market trades through the price
default early conservative model

QUEUE

estimate quantity ahead at order placement
require subsequent executions to exhaust that queue
preferred advanced model

The reports must show results under both optimistic and conservative fill assumptions.

A strategy that is profitable only under `TOUCH` but loses under `QUEUE` is not a live candidate.

Venue fee assumptions must come from current market/venue information wherever possible. Polymarket's current API includes fee-rate and tick-size market interfaces, so these should be fetched rather than guessed. ³⁵

All paper positions must settle from actual final market outcomes.

Do not infer resolution from the model.

Do not infer sports results from a tweet.

Use the venue's official market status/resolution.

Replay must be deterministic.

Given:

```
same data
same strategy version
same parameters
same random seed
same execution model
```

MarketLab must reproduce the same orders and P&L.

Clock access inside strategies must be dependency-injected. No strategy can call `wall-clock datetime.now()` directly during replay.

Strategy laboratory and market-specific experiments

Every strategy shall be a small independent plugin implementing roughly:

```

class Strategy:
    def on_market_update(self, event): ...
    def on_trade(self, event): ...
    def on_news(self, event): ...
    def on_social(self, event): ...
    def on_external_price(self, event): ...
    def on_timer(self, event): ...
    def generate_intents(self) -> list[OrderIntent]: ...

```

No strategy receives direct API credentials.

No strategy calls a broker SDK.

No strategy may bypass risk controls.

The required initial strategy tournament is below.

Market-implied baseline.

Before inventing AI, record the simplest possible forecast:

```
p_yes = current midpoint
```

This produces no trade by itself but establishes whether any sophisticated model actually improves probability accuracy.

Also track:

```

best bid
best ask
midpoint
last trade
spread
weighted mid

```

Every AI and quantitative probability model must beat a market baseline in some meaningful out-of-sample sense before claiming predictive value.

Binary complement/parity strategy.

For logically complementary YES/NO contracts, test executable inconsistencies after fees.

Conceptually:

```
cost_buy_yes + cost_buy_no < guaranteed_payout - all_costs
```

or equivalent sell-side constructions where venue mechanics permit.

Do not call the difference “profit” until:

```
both legs are simultaneously executable  
quantity exists at quoted prices  
fees included  
slippage included  
resolution semantics identical  
capital can actually be locked
```

Cross-venue relative-value strategy.

Compare:

```
Kalshi  
vs  
Polymarket Global read-only  
vs  
Polymarket US  
vs  
sportsbook reference probability  
vs  
external quantitative estimate
```

Because the user's location currently cannot open new positions on global Polymarket, a global Polymarket discrepancy may serve as a **signal** to trade an equivalent legal venue rather than as one leg of a global Polymarket execution. ³⁵

Compute:

```
kalshi_mid  
poly_global_mid  
poly_us_mid  
reference_probability  
  
kalshi_minus_poly  
kalshi_minus_consensus  
poly_us_minus_consensus
```

Create paper experiments by minimum discrepancy:

1%
2%
3%
5%
7.5%
10%

Do not optimize all thresholds on the same future period.

Prediction-market market-making strategy.

Adapt Avellaneda-Stoikov-style inventory control to bounded binary claims. 36

State:

inventory_yes
inventory_no
cash
time_to_close
spread
recent_volatility
book_depth
fill_rate

Conceptually calculate:

```
fair_probability = model or market midpoint  
reservation_price = fair_probability - inventory_penalty  
desired_spread = base_spread + volatility_adjustment +  
adverse_selection_adjustment
```

Quote only when expected captured spread comfortably exceeds expected fees and adverse selection.

Test:

one-sided maker
two-sided maker
inventory-neutral
inventory-skewed
news-pause maker

```
high-liquidity only
low-volatility only
```

Automatically cancel maker orders around stale-data conditions, market-status changes and high-impact news.

Order-book imbalance strategy.

Calculate at several depth levels:

```
bid_size / (bid_size + ask_size)
```

and richer versions:

```
top_level_imbalance
top_3_levels
top_5_levels
weighted_depth_imbalance
microprice
spread
trade_aggressor_flow
short_term_trade_intensity
```

Test both momentum and reversal interpretations.

This is evidence class E until MarketLab itself proves otherwise.

Time-series momentum.

Use academically motivated momentum as a baseline, but adapt it separately to each asset type rather than assuming one lookback works universally. Historical literature supports the broad phenomenon across conventional liquid markets, not a universal prediction-market parameter set. ³⁷

BTC reference variants:

```
1m return
5m return
15m return
1h return
4h return
24h return

volatility-adjusted return
```

```
moving-average slope  
breakout distance
```

Prediction-contract variants:

```
contract-price momentum  
contract-volume momentum  
spread-adjusted momentum  
external-underlying momentum
```

Test:

```
momentum continuation  
momentum with volatility filter  
momentum with book confirmation  
momentum excluding news spikes
```

Short-horizon mean reversion.

Use:

```
z-score of short-term returns  
deviation from rolling VWAP  
deviation from market consensus  
temporary cross-venue divergence
```

Run separately from momentum. Do not blend until the experiment phase demonstrates when each works.

Pairs/statistical relative value for stocks.

Create candidate stock pairs based on sector/economic relationship, then test cointegration/relative-deviation strategies.

Gatev-style historical pairs evidence justifies inclusion as a baseline, but current data must decide whether it survives costs now. ³⁸

Experiments:

```
distance pairs  
correlation pairs  
cointegration pairs  
sector ETF vs component
```


dual-class shares where applicable
stock vs sector ETF residual

No live stock execution is required in the first milestone. Stocks can initially be a signal research layer whose forecasts feed prediction markets.

BTC event-market strategy.

The user's requested horizons should be represented as separate universes where available:

BTC_1M
BTC_5M
BTC_15M
BTC_1H
BTC_DAILY

Do not assume all venues list all horizons.

For each event, estimate:

current BTC spot
strike/threshold
time_remaining
realized volatility
short-term momentum
market contract probability
external consensus probability

Strategy variants:

market-only momentum
spot-momentum prediction
volatility-implied probability
cross-venue discrepancy
book-imbalance confirmation
news-filtered
copy-trader-confirmed
ensemble

The engine should intentionally test whether the 1-minute horizon is too latency-sensitive for a home/local setup. A losing result is useful.

Sports strategy family.

Polymarket's API has dedicated sports metadata and realtime sports interfaces, and the optional Odds API can provide bookmaker reference lines. ³⁹

Sports experimentation must include:

```
prediction-market probability only
sportsbook consensus probability
vig-removed sportsbook consensus
prediction market vs sportsbook disagreement
pregame line movement
market momentum
copy-trader sports specialists
simple Elo/rating baseline where enough data exists
injury/news-adjusted local AI
```

Never compare American odds directly to prediction-market probabilities without converting them and, for sportsbook consensus, accounting for bookmaker margin.

A sports strategy must stop using old pregame state once a contest goes live.

Polymarket documentation notes lifecycle behavior around sports markets, including order handling at official game starts, reinforcing the need for explicit game-state synchronization. ⁴⁰

Politics strategy family.

Create:

```
poll/market consensus
cross-market probability
news-event
speech/social
copy-politics-leader
market momentum
late-resolution
```

Every political market must preserve its exact resolution source and deadline.

Do not infer an election outcome from unofficial social reports when settlement depends on a defined authority.

Economics strategy family.

Connect prediction contracts to:

```
FRED/ALFRED
scheduled economic releases
Fed communications/news
cross-venue probabilities
Treasury/rates reference data when available
```

Critical backtest rule:

use information that existed at the historical timestamp, not later revised economic data.

FRED/ALFRED-style point-in-time handling is specifically valuable because revisions can otherwise leak future information. ⁴¹

News-probability strategy.

Given recent point-in-time articles, the local model generates:

```
{
  "p_yes": 0.63,
  "confidence": 0.58,
  "abstain": false,
  "evidence_ids": ["news:abc", "news:def"],
  "contradictions": [],
  "information_cutoff": "2026-09-04T14:30:00Z"
}
```

Trade only when:

```
model_edge =
  model_probability
  - executable_market_price
  - fees
  - slippage_buffer
  - uncertainty_buffer

model_edge > configured_threshold
```

Test thresholds.

Trump/public-statements stock tracker.

Create a dedicated signal called:

TRUMP_PUBLIC_STATEMENT

but make the architecture generic enough for any configured public figure.

Sources:

official X API when credentials exist
GDELT reporting
licensed/public feeds
other platform interfaces only where permitted

The adapter shall detect:

post_timestamp
source
author
raw_text_hash
mentioned_company
mentioned_ticker
mentioned_person
sector
policy_topic
sentiment
action_type
novelty
market_hours_state

Examples of `action_type`:

praise
criticism
tariff
contract
regulatory
executive_action
foreign_policy
tax
subsidy
sanctions
antitrust
misc

The first version must **not** translate:

```
positive sentiment -> BUY  
negative sentiment -> SELL
```

That is too naive.

Instead, create event studies.

For every detected statement calculate forward returns:

```
1 minute  
5 minutes  
15 minutes  
1 hour  
market close  
1 trading day  
3 trading days  
5 trading days
```

Against:

```
raw stock return  
SPY-adjusted return  
sector-ETF-adjusted return
```

After enough observations, determine which categories actually show repeatable behavior.

The tracker can then become a trading strategy only if the out-of-sample study supports it.

SEC/public-company transaction tracker.

Use EDGAR for company events and Form 4 insider filings where applicable. SEC APIs are public, no-key JSON interfaces and update throughout the day. ²⁴

Track:

```
Form 4  
8-K  
10-Q  
10-K  
13D/13G where relevant  
material filing keywords  
insider purchase
```

```
insider sale
officer/director role
filing lag
```

Insider transactions need context. Routine scheduled sales should not automatically equal bearish signals.

Congressional transaction data requires a legal-use gate.

This PRD does **not** tell Claude to scrape the House financial-disclosure database into a commercial trading engine. The official House disclosure search page explicitly warns that use of those financial-disclosure statements for a commercial purpose is unlawful, subject to the statutory exceptions it identifies. House members' reportable securities transactions can be filed as late as 45 days after the transaction, so they are also inherently delayed rather than real-time trade alerts. ⁴²

The Senate likewise describes Periodic Transaction Reports as disclosures for securities transactions above the reporting threshold and permits reporting up to 45 days after the transaction. ⁴³

Therefore:

```
DO:
    support a LicensedPoliticalTradesProvider interface
    ingest a dataset only when its license explicitly permits the intended use
    preserve actual transaction_date and publication_date separately
    treat disclosure lag as part of the simulation

DO NOT:
    scrape House filings into an automated money-making strategy by default
    pretend politician disclosures are real-time
    backtest from transaction date if the public did not learn about it until
    later
```

A “politician trade” can enter a historical strategy only at the timestamp the data became publicly available to the system.

This is an essential anti-look-ahead rule.

Polymarket copy trader.

This is one of the highest-priority experiments because the public leaderboard and user-history API now make it technically straightforward. ⁴⁴

Discovery workflow:

```
query official leaderboard
snapshot top traders
fetch public historical activity
calculate trader statistics
classify category specialization
freeze candidate set
observe future actions
simulate follower execution after realistic delay
score forward follower performance
```

Create latency arms:

```
0 sec theoretical
1 sec
5 sec
15 sec
30 sec
2 min
5 min
10 min
```

The zero-second result is diagnostic only.

Copying at the original trader's execution price is prohibited.

Follower execution must use the book available **after MarketLab observed the trade plus configured follower delay**.

Copy strategies:

```
COPY_RAW
    follow every qualifying action

COPY_CATEGORY_SPECIALIST
    follow only in trader's best category

COPY_HIGH_CONVICTION
    follow only unusually large positions

COPY_CONSENSUS
    trade only when several tracked winners align

COPY_EARLY
    follow initial entry only
```

COPY_SCALE_IN
copy proportional changes

COPY_FADE
deliberately take opposite side
research control group

The fade control matters. It determines whether the apparent copy edge is actually informative.

Trader selection should penalize:

very short history
one giant lucky trade
extreme concentration
recent performance collapse
low follower liquidity
huge entry-price advantage that disappears after delay

Do not rank traders by raw win rate alone.

A trader can win 90% of trades and still be unprofitable if the losses are badly asymmetric.

Copy basket.

Instead of one guru:

select 5–20 validated public traders
weight by forward-validated reliability
cap each trader
cap correlated event exposure
require enough market liquidity

This implements the user's “basket trade, copy known wins” idea while reducing dependence on one account.

Consensus-of-winners strategy.

For each market:

market price
Kalshi price
Polymarket price


```
top-trader aggregate
sportsbook/reference probability
quant model
local-AI probability
news probability
```

Construct a candidate consensus probability.

Do not assign weights manually forever. Track each signal's historical calibration and forward performance.

Weather strategy.

For relevant weather markets, use official NWS forecasts/observations where geographically applicable. ²⁸

Test:

```
official forecast vs market
forecast changes
ensemble of forecast updates
market reaction lag
copy-weather specialists
```

Weather is also a useful category for copy-trader testing because Polymarket's leaderboard explicitly supports a weather category. ³²

Simulation, statistics, bankroll survival, and final-algorithm selection

MarketLab exists to answer:

“What actually works after costs, in the future, when we could really have placed the trade?”

Every strategy report must separate:

```
IN_SAMPLE
VALIDATION
FORWARD_PAPER
LIVE
```

Never combine them into one attractive equity curve.

Point-in-time discipline.

At decision time `T`, a strategy may access only records with:

```
available_to_system_at <= T
```

For every external record store:

```
event_time
published_time
first_seen_time
ingested_time
```

The field that matters to the strategy is generally `first_seen_time`, not the date described in the content.

Example:

```
Politician transaction:
transaction_date = July 1
disclosed_date   = August 10
ingested_date    = August 10
```

A July backtest may **not** use it.

The same principle applies to:

```
economic revisions
late news articles
updated injury reports
final poll averages
later-edited social content
public trader discovery
```

TradingAgents' recent work correcting point-in-time and look-ahead issues underscores how easy these errors are even in sophisticated open-source agent frameworks. ⁴⁵

Core prediction metrics.

For every strategy that emits probabilities, record:

```
Brier score
log loss
```

calibration curve
forecast count
mean predicted probability
actual resolution frequency
market probability at forecast
improvement vs market

Financial P&L alone is insufficient because a probability model can be good but traded poorly, or vice versa.

Core trading metrics.

Every experiment must calculate:

starting bankroll
ending bankroll
realized P&L
unrealized P&L
net P&L
gross P&L
fees
estimated slippage
turnover
number of trades
number of resolved trades
winning trades
losing trades
win rate
average winner
average loser
expectancy per trade
profit factor
maximum drawdown
drawdown duration
largest single loss
largest single win
average exposure
maximum exposure
average holding period
fill ratio
maker fill ratio
cancel ratio
rejected order count
stale-data skip count
risk-gate skip count

For high-frequency strategies also calculate:

quoted edge
edge at arrival
edge after 1 sec
edge after 5 sec
realized edge
adverse-selection loss
fill-adjusted return

For copy trading:

source_trader pnl
theoretical follower pnl
1s follower pnl
5s follower pnl
30s follower pnl
2m follower pnl
actual fill rate
price disadvantage

For cross-market strategies:

displayed discrepancy
executable discrepancy
post-fee discrepancy
captured discrepancy
leg failure rate

For news/social:

source latency
classifier latency
LLM latency
signal latency
entry latency
topic
novelty
confidence
forward returns

Closing-price diagnostic.

Where meaningful, calculate whether the entry moved in the strategy's predicted direction before close.

A strategy repeatedly buying at 52¢ and seeing the market later trade at 58¢ may have signal quality even if a few final binary outcomes happened to lose.

The actual settlement still determines realized P&L, but price-path information helps distinguish signal quality from outcome variance.

Benchmark everything.

Each strategy should compete against relevant controls:

```
do nothing
market midpoint
random direction at same trade frequency
random timing
opposite/fade signal
simple momentum
simple mean reversion
equal-weight trader copy
```

AI earns its place only by outperforming cheap baselines out of sample.

Bootstrap uncertainty.

Reports should estimate uncertainty around:

```
mean return per trade
expectancy
Brier improvement
copy-trader alpha
```

A strategy with:

```
3 trades
3 wins
+80%
```

is interesting but not promoted.

Minimum evidence rules.

Suggested defaults, configurable:

Fast strategies:

```
>= 200 paper orders
>= 100 resolved/closed trades where applicable
>= 14 days forward observation
positive after conservative costs
```

Medium-frequency:

```
>= 75 independent trades
>= 30 calendar days
```

Slow event strategies:

```
>= 30–50 independent resolved events when practical
```

These are engineering promotion thresholds, not universal statistical laws.

Do not treat 500 trades on the same correlated election event as 500 independent observations.

Experiment sweeper.

For each strategy:

```
base variant
small conservative parameter grid
```

Avoid thousands of hyperparameter combinations initially.

Example BTC momentum sweep:

```
lookback:
- 60s
- 300s
- 900s
- 3600s

entry_edge:
- 0.02
- 0.04
- 0.06

max_spread:
```

- 0.02
- 0.04

volatility_filter:

- false
- true

Every added dimension creates more chances to discover random noise.

Record the total number of variants tried.

Champion/challenger system.

Statuses:

IDEA
BACKTESTING
PAPER
QUALIFIED
CHAMPION
DEGRADED
DISABLED
LIVE_SMALL
LIVE_PROVEN

Promotion:

IDEA
-> passes deterministic tests
BACKTESTING
-> acceptable historical results
PAPER
-> forward positive after costs
QUALIFIED
-> sufficient sample and risk profile
CHAMPION

A champion can later become:

DEGRADED
DISABLED

Past success gives no permanent immunity.

"If it isn't broke, don't fix it" implementation.

Once a champion performs well:

Do **not** continuously retune it.

Freeze:

```
strategy version
parameters
feature definitions
data pipeline
```

Run new improvements as challengers.

The champion changes only when a challenger wins forward.

That is the formal software equivalent of the user's principle.

Virtual \$50 leagues.

Every strategy variant receives a standardized virtual bankroll:

```
$50
```

Leaderboard output:

```
rank
strategy
category
days_alive
initial_capital
equity
net_pnl
roi
max_drawdown
trade_count
profit_factor
calibration
status
```

Also run a normalized risk leaderboard because an aggressive strategy should not win solely by risking its entire \$50.

Death rule.

A paper sleeve is `DEAD` when it can no longer satisfy minimum trade requirements or reaches a configured near-zero threshold.

Do not delete it.

Example:

```
Experiment PM_MOMENTUM_V7
Started: $50
Ended:    $1.42
Status:   DEAD
```

A fresh parameter variant starts:

```
PM_MOMENTUM_V8
Starting bankroll: $50
```

The research database remembers both.

Live \$50 risk policy.

Suggested initial live constraints:

```
live:
  initial_capital: 50.00

  leverage: false
  borrowing: false
  martingale: false

  max_single_event_loss_pct: 0.04
  max_strategy_exposure_pct: 0.20
  max_category_exposure_pct: 0.30
  max_correlated_cluster_pct: 0.30

  daily_loss_pause_pct: 0.10
  total_drawdown_pause_pct: 0.20

  auto_replenish: false
```

If venue minimum order size would force a trade above the risk limit:

SKIP THE TRADE

Do not override risk limits merely to participate.

A daily loss pause does not falsify the user's \$50 "die with it" experiment. It allows the system to diagnose failure instead of losing the remaining capital in the same broken condition.

No automatic Kelly sizing at the beginning.

MarketLab may calculate fractional-Kelly recommendations as a research metric, but real sizing should remain hard-capped until probability calibration is demonstrated.

Overconfident probability estimates make Kelly sizing dangerously aggressive.

Final ensemble.

The end product is not necessarily one formula. It can be a constrained portfolio of independently useful components.

Candidate inputs:

```
market microstructure strategy
cross-market strategy
copy-trader consensus
BTC quantitative probability
sports consensus
news model
event/social model
market-making strategy
```

Only **QUALIFIED** strategies enter.

Initial ensemble weight logic:

```
weight increases with:
  forward expected value
  sample reliability
  calibration
  consistency

weight decreases with:
  drawdown
  correlation with other strategies
```

```
execution sensitivity
recent degradation
low liquidity
```

Hard constraint:

```
weight >= 0
```

at first.

Do not let the optimizer create an incomprehensible highly leveraged long/short portfolio simply because historical covariance suggested it.

Cap individual strategy weights.

An example first ensemble:

Copy-consensus	20%
Cross-venue relative	20%
Prediction MM	15%
BTC model	15%
Sports consensus	10%
News probability	10%
Other qualified signals	10%

Those values are placeholders; the experiment engine must earn the final weights.

A more robust final decision rule may be:

```
Take a position only when:

at least 2 independent signal families agree

AND

ensemble expected edge
  > fees
  + expected slippage
  + uncertainty buffer

AND

liquidity is sufficient
```

AND

risk gateway approves

This naturally reduces reliance on one hallucinating AI or one lucky trader.

Local AI, news intelligence, social tracking, and copy-trader reasoning

The local model is an **analyst**, not the broker.

Ollama is a natural first integration because its local API is served by default on localhost and supports programmatic model interaction. Its current API also supports structured/tool-oriented patterns. ⁴⁶

Claude should first inspect what is already installed:

```
which ollama || true
ollama list || true
curl -s http://localhost:11434/api/tags || true
```

Do not download another giant model automatically if a capable local one already exists.

Autodetect provider priority:

```
existing Ollama
existing OpenAI-compatible localhost server
existing llama.cpp server
explicit LOCAL_LLM_BASE_URL
disabled AI mode
```

Ollama's localhost API should remain bound locally rather than exposed indiscriminately; its local interface does not require the same remote authentication model as a public cloud service. ⁴⁷

The model never receives a raw trade tool.

The architecture must be:

```
DATA
↓
RETRIEVAL
↓
```

```
LOCAL MODEL
↓
STRICT JSON ASSESSMENT
↓
DETERMINISTIC VALIDATOR
↓
STRATEGY
↓
RISK GATEWAY
↓
PAPER/LIVE BROKER
```

Not:

```
LLM -> shell -> venue API
```

LLM output schema:

```
{
  "market_id": "canonical-market-id",
  "as_of": "2026-09-04T15:00:00Z",
  "question_interpretation": "...",
  "p_yes": 0.61,
  "confidence": 0.66,
  "abstain": false,
  "evidence_ids": [
    "gdelt:...",
    "sec:..."
  ],
  "supporting_facts": [],
  "contradicting_facts": [],
  "missing_information": [],
  "resolution_rule_warning": false,
  "information_cutoff": "2026-09-04T15:00:00Z"
}
```

Validate:

```
0 <= p_yes <= 1
0 <= confidence <= 1
all evidence IDs exist
no evidence timestamp > decision timestamp
```

```
market exists  
resolution interpretation matches stored rules
```

If invalid:

```
ABSTAIN
```

Do not “repair” a malformed trade recommendation silently.

AI should constantly run as a service, not constantly burn compute on every tick.

The daemon remains alive continuously.

Inference is event-triggered:

```
new high-impact news  
new SEC filing  
new social post  
large market move  
large cross-venue discrepancy  
tracked trader action  
scheduled refresh
```

For quiet markets, periodic refresh:

```
15 min  
30 min  
60 min
```

depending on market horizon.

For 1-minute BTC contracts, the LLM is unlikely to be the low-latency core. Use deterministic market/price features for micro-horizons and let AI provide event context.

Retrieval system.

For each active market maintain a small evidence bundle:

```
market rules  
latest relevant news  
recent social posts  
SEC filings
```

```
external market prices
tracked trader activity
prior model assessment
counterevidence
```

Use semantic search plus entity filters.

Do not pass the entire news archive to the model.

News deduplication.

Wire stories are repeatedly republished. Detect duplicates using:

```
canonical URL when available
headline similarity
body hash
semantic similarity
source timestamp
```

Ten copies of the same report must not count as ten independent sources.

Source weighting.

Maintain:

```
official_primary
major_wire
major_news
specialist
social_verified
social_unverified
unknown
```

An SEC filing or official league result generally deserves different evidentiary treatment from an anonymous social post.

News ingestion priority.

Recommended:

```
GDELT
official government/agency feeds
SEC
```

```
RSS feeds explicitly allowed
Alpaca news if credentials
X API
Bluesky
other licensed providers
```

GDELT provides a strong broad global backbone, while SEC gives direct primary-source corporate disclosures. ⁴⁸

Trump tracker architecture.

Maintain configuration:

```
public_figures:
  trump:
    enabled: true
    sources:
      - x
      - gdelt
    keywords:
      - tariff
      - china
      - fed
      - rates
      - oil
      - semiconductor
      - auto
      - defense
      - pharma
      - crypto
```

Ticker/entity resolution shall map organization names to symbols:

```
"Apple" -> AAPL
"Tesla" -> TSLA
"Boeing" -> BA
```

but ambiguous terms require abstention.

Track the empirical impact of each topic.

Example analytics:


```
TRUMP + TARIFF + AUTO
observations: 43
median 15m abnormal return: ...
median 1d abnormal return: ...
hit rate: ...
confidence interval: ...
```

No hard-coded assumption that a particular politician's statement is bullish or bearish.

AI disagreement is valuable.

Run optional multi-pass assessment:

```
Analyst:
    make best estimate

Skeptic:
    identify reasons estimate may be wrong

Resolver:
    output final probability
```

This idea resembles the structured role separation explored in multi-agent finance projects such as TradingAgents, though MarketLab must empirically test whether the extra inference cost adds forecast value. ⁴⁹

Also test **single-model simple prompting** as the control.

Never assume more agents means better trading.

Copy-trader AI classification.

The local model may summarize a trader's apparent style:

```
sports specialist
late favorite buyer
news trader
market maker
high turnover
long-tail bettor
crypto microstructure
event arbitrage
```

But classification must be derived from observable trades.

Do not claim a wallet is an “AI bot” merely because it trades frequently.

Some 2026 reporting and social analysis claims substantial bot representation among top Polymarket wallets, but those classifications are not equivalent to audited identity data. Treat bot labels as hypotheses, while actual public trade performance remains the primary signal. 50

Trader graph.

For every candidate trader:

```
Trader
├─ Markets
├─ Categories
├─ Entries
├─ Exits
├─ Position changes
├─ Timing
├─ P&L snapshots
└─ Social identity, when exposed publicly
```

Cluster traders by behavior.

This can reveal whether ten “different winners” are effectively following the same trade.

Viral challenge tracker.

Create:

```
social_challenge_candidates
```

Fields:

```
claim
account
platform
claimed_starting_balance
claimed_current_balance
claim_date
wallet_if_public
verified_by_market_data
```

```
verification_status
notes
```

Statuses:

```
UNVERIFIED
PARTIALLY_VERIFIED
ONCHAIN_OR_API_CONFIRMED
DISPROVEN
STALE
```

No strategy copies an `UNVERIFIED` challenge candidate.

This fulfills the “find growing \$100 to \$1m accounts” request without turning marketing stories into supposed quantitative evidence.

Claude terminal build specification and definition of done

This section is written directly for the coding agent.

Claude: treat everything below as implementation requirements, not as optional conceptual suggestions.

The task is not complete when you generate folders.

The task is not complete when a README exists.

The task is not complete when one API call works.

The required endpoint is a **running terminal-only paper-trading research system using real market data**.

Start by inspecting the environment:

```
set -e

uname -a || true
pwd
git status || true
python3 --version || true
uv --version || true
git --version || true
```

```
which ollama || true  
ollama list || true
```

Do not destroy an existing repository.

If working inside an existing project, inspect it before writing files.

Create a clean Python environment.

A recommended bootstrap flow:

```
uv venv  
source .venv/bin/activate  
uv sync
```

Create `.env.example`, never `.env` secrets committed to Git.

At minimum support:

```
MARKETLAB_MODE=PAPER  
  
KALSHI_API_KEY_ID=  
KALSHI_PRIVATE_KEY_PATH=  
  
POLYMARKET_US_KEY_ID=  
POLYMARKET_US_SECRET_KEY=  
  
ALPACA_API_KEY=  
ALPACA_SECRET_KEY=  
  
X_BEARER_TOKEN=  
  
THE_ODDS_API_KEY=  
FRED_API_KEY=  
  
LOCAL_LLM_PROVIDER=auto  
LOCAL_LLM_BASE_URL=  
LOCAL_LLM_MODEL=  
  
LIVE_TRADING_ENABLED=NO
```

Add:

```
.env  
*.pem  
*.key  
credentials*
```

to `.gitignore`.

Private keys must never appear in structured logs.

Implement the adapters in dependency order.

First:

```
Kalshi market discovery REST  
Kalshi production books  
Kalshi trades  
Polymarket global Gamma  
Polymarket global CLOB read  
Polymarket global Data API  
Polymarket leaderboard  
Polymarket US public SDK
```

Then:

```
GDELT  
SEC  
local AI
```

Then:

```
X  
Bluesky  
Alpaca  
sports odds  
FRED  
NWS
```

Paid/credentialed optional adapters must not prevent the core system from booting.

If X credentials are absent:

```
WARN x source unavailable  
CONTINUE
```

If Kalshi private credentials are absent but public REST market data works:

```
run REST ingestion  
disable authenticated WS/live  
CONTINUE PAPER
```

If Polymarket US credentials are absent:

```
use public market data  
disable live  
CONTINUE PAPER
```

The official Kalshi environments use separate production and demo credentials, so configuration must never silently send demo credentials to production or vice versa. ⁵¹

Polymarket safety check.

At startup perform global geoblock detection.

If blocked:

```
POLYMARKET_GLOBAL_EXECUTION=false
```

It must remain false.

Do not provide a VPN/proxy workaround.

Current official documentation lists the U.S. as close-only for global Polymarket new positions. ³⁵

Current SDK requirement.

Do not install legacy global Polymarket V1 clients simply because an old tutorial says so. Current integration must use current documented APIs/SDKs, while Polymarket US should use its official `polymarket-us` package where practical. ⁵²

Create CLI commands.

Required:

```
marketlab doctor
marketlab sources
marketlab markets
marketlab markets --venue kalshi
marketlab markets --venue poly-global
marketlab markets --venue poly-us

marketlab ingest
marketlab ingest --once
marketlab ingest --daemon

marketlab trader-discover
marketlab trader-show <wallet>

marketlab ai doctor
marketlab ai assess <market-id>

marketlab paper start
marketlab paper status
marketlab paper leaderboard
marketlab paper stop

marketlab experiments list
marketlab experiments show <id>
marketlab experiments compare <id> <id>

marketlab report daily
marketlab report strategies
marketlab report categories
marketlab report traders
marketlab report risk

marketlab replay <date-or-dataset>
marketlab live doctor
```

`marketlab live start` must not work merely because credentials exist.

Require all of:

```
LIVE_TRADING_ENABLED=YES_I_ACCEPT_REAL_LOSS
valid venue credentials
venue eligibility success
all tests green
risk config loaded
```

```
no stale market feed
explicit CLI confirmation
```

Prefer requiring an additional command-line string:

```
marketlab live start --acknowledge-real-money-risk
```

`marketlab doctor` **output should resemble:**

```
MarketLab Doctor
=====

Database                OK
Parquet storage         OK
Clock                   OK

Kalshi REST             OK
Kalshi WebSocket        AUTHENTICATED
Kalshi Live Orders      DISABLED

Polymarket Global Data  OK
Polymarket Global Geo   BLOCKED FOR NEW US ORDERS
Polymarket Global Trade DISABLED

Polymarket US Public    OK
Polymarket US Trading   NO CREDENTIALS

GDELT                   OK
SEC EDGAR                OK
X                        NO CREDENTIALS
Bluesky                  OK
NWS                      OK

Ollama                   OK
Model                    <detected model>

Mode                     PAPER
Real-money trading       HARD DISABLED
```

Market ingestion acceptance test.

After startup, the database must contain real active market rows.

Test:


```
marketlab ingest --once
marketlab markets --limit 20
```

Failure condition:

```
zero markets
```

must produce an explicit error rather than pretending setup succeeded.

Leaderboard acceptance test.

Query the official Polymarket trader leaderboard and persist a dated snapshot. The current official endpoint supports category/time-period/P&L filters and returns trader profile addresses plus P&L/volume fields. ³²

Run:

```
marketlab trader-discover
marketlab report traders
```

Expected report:

```
Snapshot time
Category
Rank
Wallet
Username
Period P&L
Volume
Forward-tracking status
```

Do not copy anyone immediately.

Mark:

```
DISCOVERED
```

and begin forward tracking.

PaperBroker acceptance tests.

Unit test:

```
book asks:
0.51 x 10
0.52 x 20
0.55 x 100
```

```
buy 25
```

Expected:

```
10 @ 0.51
15 @ 0.52
```

not:

```
25 @ 0.51
```

Test partial fills.

Test insufficient liquidity.

Test stale books.

Test limit queue rules.

Test cancellation.

Test market settlement.

Test YES/NO accounting.

Test fees.

Test restart/recovery.

No-look-ahead test.

Create:

```
news published 10:05
historical clock 10:04
```

Strategy must not see it.

At:

10:05+

it may.

Create equivalent tests for:

```
political disclosure
SEC filing
economic revision
trader-discovery timestamp
sports injury
```

Cross-market matching acceptance test.

Create near-identical but non-equivalent rules.

The matcher must reject them.

Create truly equivalent synthetic rules.

The matcher may approve.

No `arbitrage` strategy may bypass this module.

Initial mandatory strategies.

Implement before adding exotic AI:

```
baseline_market
momentum
mean_reversion
book_imbalance
binary_parity
cross_venue_relative_value
market_maker
copy_trader
sports_consensus
btc_event
news_probability
public_statement_event
```

Each must have at least one deterministic unit test.

Experiment matrix.

Create generated variants from YAML.

Example:

```
strategies:

  momentum:
    enabled: true
    universes:
      - prediction_crypto
      - btc_reference
    variants:
      lookback_seconds: [60, 300, 900, 3600]
      threshold: [0.01, 0.02, 0.04]

  book_imbalance:
    enabled: true
    variants:
      levels: [1, 3, 5]
      threshold: [0.60, 0.70, 0.80]

  copy_trader:
    enabled: true
    variants:
      follower_delay_seconds: [1, 5, 30, 120]
      mode:
        - raw
        - specialist
        - consensus

  cross_venue:
    enabled: true
    variants:
      min_edge: [0.02, 0.03, 0.05]
```

Every variant gets a separate virtual \$50 sleeve.

Reporting output.

Terminal report:

STRATEGY LEAGUE

Rank	Strategy	Equity	P&L	DD	Trades	Status
1	COPY_CONSENSUS_5S	\$58.20	+8.20	-4.1%	92	PAPER
2	CROSS_VENUE_3PCT	\$55.42	+5.42	-2.3%	41	PAPER
3	MM_INVENTORY_V2	\$52.81	+2.81	-1.9%	301	PAPER
...						
42	BTC_1M_MOMENTUM	\$21.40	-28.60	-61.7%	801	DEAD

Also:

BEST BY CATEGORY
BEST RISK-ADJUSTED
BEST CALIBRATED
MOST CONSISTENT
WORST DRAWDOWN
WORST EXECUTION SLIPPAGE
MOST OVERFIT
MOST LATENCY-SENSITIVE

The report must celebrate losers as useful findings.

Daily JSON report.

Persist machine-readable results:

```
{
  "date": "2026-09-04",
  "strategies_running": 84,
  "strategies_alive": 79,
  "strategies_dead": 5,
  "best_net_pnl": {},
  "best_risk_adjusted": {},
  "largest_drawdown": {},
  "data_health": {},
  "venue_health": {}
}
```

Crash recovery.

On restart:

```
reload open paper orders
reload positions
reload experiment states
reconnect feeds
reconcile market statuses
resume
```

Never create a fresh paper bankroll because the process restarted.

Daemon design.

The service should run foreground normally:

```
marketlab run
```

For sustained machine operation, provide a terminal-installable user service:

```
scripts/install_service.sh
```

that generates a `systemd --user` service where systemd is available.

Do not require a GUI.

Expected service commands:

```
systemctl --user start marketlab
systemctl --user stop marketlab
systemctl --user restart marketlab
systemctl --user status marketlab
journalctl --user -u marketlab -f
```

If systemd is unavailable, document an equivalent foreground/tmux launch path.

Health watchdog.

Every feed has:

```
last_message_at
latency
error_count
```

```
reconnect_count
status
```

States:

```
HEALTHY
DEGRADED
STALE
DOWN
```

If required market data becomes **STALE**:

```
new trading intents disabled
resting live orders canceled where safe/appropriate
alert written
```

Do not trade blind.

Observability.

Use structured JSON logs plus readable console output.

Every trade decision must be reconstructable:

```
why strategy acted
features used
timestamp
market state
external evidence
LLM output if any
risk decision
order intent
fill result
eventual outcome
```

There shall be no unexplained:

```
BUY because AI said so
```

record.

Security.

Live signing components must be isolated from research agents.

The local LLM cannot:

```
read private keys
read .env secrets
invoke arbitrary shell
change live risk limits
activate live mode
withdraw funds
```

For Polymarket systems where current session-key capabilities are applicable, prefer scoped/time-limited keys over broad wallet authority because current platform documentation supports constrained session-style access patterns. ⁵³

Test gates.

Before `paper start`:

```
unit tests pass
database migrations pass
at least one venue data source healthy
```

Before `LIVE_SMALL`:

```
all unit tests pass
all integration tests pass
paper broker verified
strategy qualifies
risk gateway passes
data feed healthy
venue terms/eligibility pass
credentials scoped
paper/live code-path parity test passes
```

Real-money promotion should be intentionally boring.

First live phase:

```
total capital = $50
only one or a few highest-confidence qualified strategies
tiny positions
```


no leverage
no martingale
no automatic replenishment
full audit trail

Do not launch 80 experimental strategies simultaneously against the same \$50 real bankroll.

The experiments remain paper.

Live is the championship round.

Definition of done for the initial Claude execution.

Claude should not claim completion until all of the following are true:

Requirement	Mandatory
Repository/environment initialized	Yes
Terminal CLI works	Yes
Kalshi production market ingestion	Yes
Local Kalshi PaperBroker	Yes
Polymarket global read-only ingestion	Yes
Global Polymarket geoblock enforced	Yes
Official Polymarket leaderboard ingestion	Yes
Public trader forward tracker	Yes
Polymarket US public adapter	Yes
Local AI autodetection	Yes
GDELT news ingestion	Yes
SEC ingestion	Yes
BTC strategy variants	Yes
Sports strategy framework	Yes
Politics/news strategy framework	Yes
Trump/public-statement event tracker framework	Yes
Momentum baseline	Yes
Mean-reversion baseline	Yes

Requirement	Mandatory
Book imbalance	Yes
Market maker	Yes
Cross-market comparator	Yes
Copy trader	Yes
\$50 virtual sleeves	Yes
Fee/slippage/latency-aware execution	Yes
Immutable experiment IDs	Yes
Persistent statistics	Yes
Terminal leaderboard	Yes
Restart recovery	Yes
Live trading disabled by default	Yes
Test suite green	Yes
Continuous paper process successfully started	Yes

The first runnable command sequence Claude should aim to make succeed is:

```
uv sync

uv run marketlab doctor

uv run marketlab ingest --once

uv run marketlab markets --venue kalshi --limit 10
uv run marketlab markets --venue poly-global --limit 10
uv run marketlab markets --venue poly-us --limit 10

uv run marketlab trader-discover
uv run marketlab report traders

uv run marketlab ai doctor

uv run pytest
uv run ruff check .
uv run mypy marketlab

uv run marketlab paper start --bankroll-per-strategy 50
```

Then verify:

```
uv run marketlab paper status
uv run marketlab paper leaderboard
uv run marketlab report strategies
```

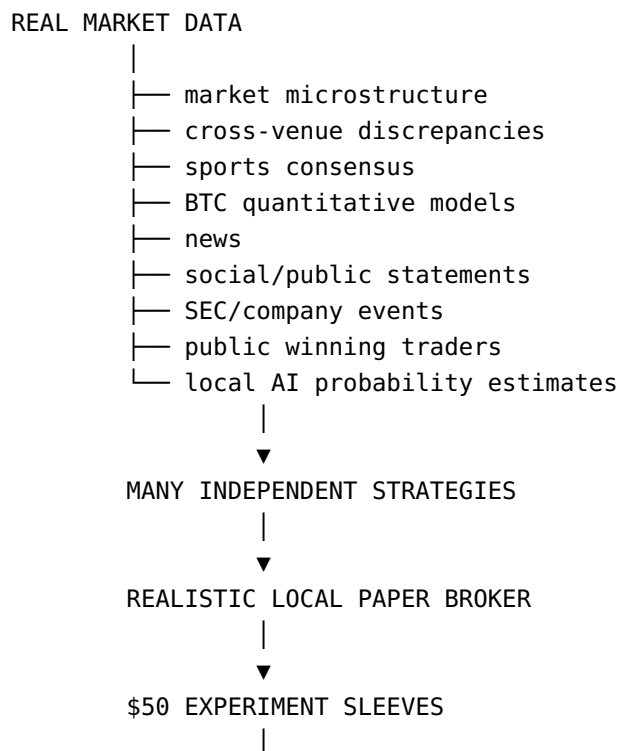
A successful initial run should visibly show that real production market information is arriving while every order remains local:

MODE: PAPER

```
Kalshi Market Data:      LIVE
Polymarket Global Data:  LIVE
Polymarket Global Orders: DISABLED
Polymarket US Data:      LIVE/AVAILABLE
Real Orders Submitted:   0

Paper Strategies:        running
Virtual Bankroll:        $50 per experiment
```

Finally, the architecture should make the eventual final algorithm a **product of evidence**, not a predetermined favorite:





That is the intended final system: not an AI blindly placing bets, and not a hand-selected backtest winner, but a continuously running research machine in which conventional quant algorithms, prediction-market microstructure, public winner replication, cross-market price comparison, world news, social events, sports information and a local language model all compete under identical execution and accounting rules.

Kalshi supplies the production exchange data and an eventual legitimate live execution path; its demo remains useful only for API mechanics, while local paper execution uses live books. ⁵⁴ Polymarket global supplies unusually rich read-only market and public-trader intelligence—including current leaderboards, profiles, activity and trade history—while its current U.S. geoblock is honored rather than bypassed; Polymarket US provides the separate U.S.-oriented public/trading SDK path. ⁵⁵ GDELT, SEC EDGAR, FRED, NWS, official/authorized social interfaces and optional stock/sports feeds provide the external information layer. ⁵⁶

Most importantly, academic momentum and pairs research, foundational market-making theory, current prediction-market arbitrage research, major open-source trading frameworks, public Polymarket winners, and AI-agent repositories enter MarketLab as **testable hypotheses with different evidence grades—not as promises of profit.** ⁵⁷

The final rule Claude should encode into the project is:

Never promote a strategy because it sounds smart.

Never promote it because GitHub likes it.

Never promote it because somebody posted a huge return.

Never promote it because an LLM is confident.

Never promote it because the backtest is beautiful.

Promote it because:

it was defined before the forward test,
it saw only information available at the time,
it traded against realistic executable prices,
it survived fees, spread, latency and poor fills,
it repeated its edge across enough observations,
it did not depend on one lucky outcome,
it fits within the \$50 risk budget,
and it continues to work after promotion.

If it keeps working, freeze it and let challengers try to beat it.

If it breaks, demote it.

Never erase the loss.

1 19 51 54 https://docs.kalshi.com/getting_started/api_environments

https://docs.kalshi.com/getting_started/api_environments

2 <https://docs.polymarket.com/api-reference/predictions/overview>

<https://docs.polymarket.com/api-reference/predictions/overview>

3 35 <https://docs.polymarket.com/api-reference/geoblock>

<https://docs.polymarket.com/api-reference/geoblock>

4 <https://github.com/Polymarket/polymarket-us-python>

<https://github.com/Polymarket/polymarket-us-python>

5 Test In The Demo Environment

https://docs.kalshi.com/getting_started/demo_env?utm_source=chatgpt.com

6 37 57 Time series momentum

https://www.sciencedirect.com/science/article/pii/S0304405X11002613?utm_source=chatgpt.com

7 38 Pairs Trading: Performance of a Relative-Value Arbitrage Rule

https://academic.oup.com/rfs/article-abstract/19/3/797/1646694?utm_source=chatgpt.com

8 36 High-frequency trading in a limit order book

https://ideas.repec.org/a/taf/quantf/v8y2008i3p217-224.html?utm_source=chatgpt.com

9 Arbitrage in Prediction Markets - DROPS - Schloss Dagstuhl

https://drops.dagstuhl.de/storage/00lipics/lipics-vol354-aft2025/html/LIPICs.AFT.2025.27/LIPICs.AFT.2025.27.html?utm_source=chatgpt.com

10 <https://github.com/hummingbot/hummingbot>

<https://github.com/hummingbot/hummingbot>

11 <https://github.com/freqtrade/freqtrade>

<https://github.com/freqtrade/freqtrade>

12 <https://github.com/mementum/backtrader/pull/447>

<https://github.com/mementum/backtrader/pull/447>

- 13 https://github.com/nautechsystems/nautilus_trader
https://github.com/nautechsystems/nautilus_trader
- 14 45 <https://github.com/tauricresearch/tradingagents>
<https://github.com/tauricresearch/tradingagents>
- 15 <https://fingpt.io/>
<https://fingpt.io/>
- 16 <https://github.com/virattt/ai-hedge-fund>
<https://github.com/virattt/ai-hedge-fund>
- 17 <https://github.com/polymarket/agents>
<https://github.com/polymarket/agents>
- 18 52 **Migrating to CLOB V2**
https://docs.polymarket.com/v2-migration?utm_source=chatgpt.com
- 20 **Quick Start: Authenticated Requests**
https://docs.kalshi.com/getting_started/quick_start_authenticated_requests?utm_source=chatgpt.com
- 21 **Rate Limits and Tiers**
https://docs.kalshi.com/getting_started/rate_limits?utm_source=chatgpt.com
- 22 **Rate Limits**
https://docs.polymarket.com/api-reference/rate-limits?utm_source=chatgpt.com
- 23 48 56 <https://www.gdeltproject.org/data.html>
<https://www.gdeltproject.org/data.html>
- 24 <https://www.sec.gov/search-filings/edgar-application-programming-interfaces>
<https://www.sec.gov/search-filings/edgar-application-programming-interfaces>
- 25 <https://docs.alpaca.markets/us/docs/real-time-stock-pricing-data>
<https://docs.alpaca.markets/us/docs/real-time-stock-pricing-data>
- 26 <https://www.alphavantage.co/>
<https://www.alphavantage.co/>
- 27 41 <https://fred.stlouisfed.org/docs/api/fred/>
<https://fred.stlouisfed.org/docs/api/fred/>
- 28 <https://www.weather.gov/documentation/services-web-API>
<https://www.weather.gov/documentation/services-web-API>
- 29 <https://the-odds-api.com/>
<https://the-odds-api.com/>
- 30 <https://developer.x.com/>
<https://developer.x.com/>
- 31 <https://docs.bsky.app/docs/advanced-guides/firehose>
<https://docs.bsky.app/docs/advanced-guides/firehose>
- 32 44 55 <https://docs.polymarket.com/api-reference/core/get-trader-leaderboard-rankings>
<https://docs.polymarket.com/api-reference/core/get-trader-leaderboard-rankings>

- 33 <https://docs.polymarket.com/api-reference/core/get-user-activity>
<https://docs.polymarket.com/api-reference/core/get-user-activity>
- 34 https://www.linkedin.com/posts/daniilkozin_polymarket-aitrading-predictionmarkets-activity-7438160024692146178-0OsQ
https://www.linkedin.com/posts/daniilkozin_polymarket-aitrading-predictionmarkets-activity-7438160024692146178-0OsQ
- 39 **Get sports metadata information**
https://docs.polymarket.com/api-reference/sports/get-sports-metadata-information?utm_source=chatgpt.com
- 40 **Markets & Events**
https://docs.polymarket.com/concepts/markets-events?utm_source=chatgpt.com
- 42 <https://disclosures-clerk.house.gov/FinancialDisclosure/ViewSearch>
<https://disclosures-clerk.house.gov/FinancialDisclosure/ViewSearch>
- 43 <https://www.ethics.senate.gov/public/index.cfm/financialdisclosure>
<https://www.ethics.senate.gov/public/index.cfm/financialdisclosure>
- 46 <https://docs.ollama.com/api/introduction>
<https://docs.ollama.com/api/introduction>
- 47 <https://docs.ollama.com/api/authentication>
<https://docs.ollama.com/api/authentication>
- 49 <https://arxiv.org/html/2412.20138v5>
<https://arxiv.org/html/2412.20138v5>
- 50 <https://www.financemagnates.com/trending/prediction-markets-are-turning-into-a-bot-playground/>
<https://www.financemagnates.com/trending/prediction-markets-are-turning-into-a-bot-playground/>
- 53 <https://docs.polymarket.com/trading/session-keys>
<https://docs.polymarket.com/trading/session-keys>